Major Project Report

PulseGrid: AI-Powered Predictive Maintenance Platform for Industrial IoT

Project Category: IoT Based / Deep Tech Based

Submitted in partial fulfilment of the requirement of the degree of

MASTER OF COMPUTER APPLICATIONS

Semester - IV

to

K.R. Mangalam University

by

Daksh Jeena (2401560053)

Aman Chourasia (2401560044)

Under the supervision of

Atisha Dahiya
(Internal Mentor)

Department of Computer Science and Engineering

School of Engineering and Technology

K.R. Mangalam University, Gurugram - 122001, India

May 2026

DECLARATION

I hereby declare that this Major Project Report entitled "PulseGrid: AI-Powered Predictive Maintenance Platform for Industrial IoT" is my original work and has not been submitted elsewhere for the award of any other degree or diploma. The information and results presented here are true and correct to the best of my knowledge.

All sources and references used during the course of this work have been duly acknowledged and cited.

(Signature)

Student Name: **Daksh Jeena**

Roll No.: **2401560053**

Date: _____

(Signature)

Student Name: **Aman Chourasia**

Roll No.: **2401560044**

Date: _____

CERTIFICATE

This is to certify that the Major Project entitled "PulseGrid: AI-Powered Predictive Maintenance Platform for Industrial IoT", submitted by **Daksh Jeena (2401560053) and Aman Chourasia (2401560044)** to K.R. Mangalam University, Gurugram, India, is a record of Bonafide project work carried out under my supervision and guidance and is worthy of consideration for the partial fulfilment of the degree of **MASTER OF COMPUTER APPLICATIONS** of the University.

(Signature of Internal Supervisor)

Name: Atisha Dahiya

Designation: _____

Date: _____

INDEX

S.No.	Content	Page No.
	Project Completion Certificate	
1	Abstract	
2	Introduction	
3	Motivation	
4	Literature Review / Comparative Work Evaluation	
5	Gap Analysis	
6	Problem Statement	
7	Objectives	
8	Tools / Platforms Used	
9	Methodology	
10	Experimental Setup	
11	Evaluation Metrics	
12	Results and Discussion	
13	Conclusion & Future Work	
14	References	
	Annexure I - Plagiarism Declaration Certificate	

ABSTRACT

Keywords: Predictive Maintenance, Industrial IoT, Machine Learning, Random Forest, Sensor Analytics, Real-Time Dashboard, FastAPI, React

Industrial machines are expensive to buy, maintain, and repair. Unplanned breakdowns can easily cost manufacturers thousands of rupees per minute and globally add up to tens of billions of dollars each year. Traditional maintenance approaches are either reactive, where machines are fixed only after they fail, or preventive, where servicing is done on a fixed schedule even if the machine is healthy. Both strategies are inefficient: the first leads to sudden downtime, while the second wastes time and resources.

This project presents PulseGrid, an end-to-end, full-stack, AI-powered predictive maintenance platform designed for Industrial IoT (IIoT) environments. PulseGrid ingests real-time sensor telemetry from simulated CNC machines and industrial equipment, processes the data through an automated cleaning and feature-engineering pipeline, and feeds it into a Random Forest classifier trained on the AI4I 2020 Predictive Maintenance Dataset, which contains 10,000 instances with 14 attributes and is available from the UCI Machine Learning Repository.

The platform comprises four major subsystems:

1. Machine Learning Pipeline - An automated scikit-learn pipeline that handles data cleaning, categorical encoding, feature engineering (temperature differential, power metric), median imputation, standard scaling, and Random Forest classification with class balancing. The model achieves 99.0% accuracy, 94.4% precision, 75.0% recall, and an F1-score of 0.836 on the test set.

2. Backend API Server - A FastAPI-based REST API with 18+ endpoints for sensor data ingestion, ML inference, alert management, AI chatbot query processing, model metrics monitoring, dataset upload with auto-retrain, sensor calibration management, and data quality analysis.

3. Real-Time Frontend Dashboard - A 7-tab React + Vite dashboard with a premium dark-mode glassmorphism UI featuring Fleet Overview, Machine Detail, Trend Analysis, Alert Center, AI Data Assistant, Dataset & Model Management, and Sensor Calibration interfaces.

4. Multi-Machine Sensor Simulator - A configurable Python simulator that generates realistic sensor data for 5 distinct machine profiles with gradual tool wear degradation, sinusoidal operational patterns, and random fault injection.

The system is containerized via Docker Compose for single-click deployment and is designed for both educational demonstration and real-world adaptation.

Chapter 1

INTRODUCTION

1.1 Background of the Project

The Fourth Industrial Revolution, commonly referred to as Industry 4.0, has fundamentally reshaped manufacturing through the convergence of physical production systems with digital technologies. At the heart of this transformation lies the Industrial Internet of Things (IIoT) - a network of interconnected sensors, actuators, and computing systems embedded within industrial machinery that continuously generates vast streams of operational data.

Modern manufacturing facilities deploy CNC (Computer Numerical Control) machines, lathes, presses, and drills that operate under extreme conditions: high rotational speeds exceeding 3,000 RPM, temperatures above 300 Kelvin, torque loads surpassing 70 Nm, and cutting tools that degrade over hundreds of operational minutes. Each of these parameters tells a story about the machine's health - subtle changes in temperature differentials, gradual increases in torque requirements, and the steady progression of tool wear all serve as early indicators of impending failure.

Historically, industrial maintenance has followed two primary strategies:

Reactive Maintenance (Run-to-Failure): Equipment is used until it breaks down. While this minimizes upfront maintenance costs, unplanned downtime results in production losses, cascading failures, quality defects, and safety hazards. Studies estimate that reactive maintenance costs 3-10x more than preventive approaches due to emergency repairs, expedited parts procurement, and lost production.

Preventive Maintenance (Time-Based): Equipment is serviced on a fixed schedule, regardless of its actual condition. While this reduces unexpected failures, it leads to unnecessary maintenance on healthy machines, premature replacement of functional components, and significant wasted labour and material costs. Industry analyses suggest that 30-40% of preventive maintenance activities are performed on equipment that does not need servicing.

Predictive Maintenance (PdM) represents the evolution beyond these strategies. By continuously monitoring machine sensor data and applying machine learning algorithms to identify patterns that precede failure, PdM enables maintenance to be performed precisely

when needed - not too early (wasting resources) and not too late (causing breakdowns). Industry reports, including those by McKinsey & Company, indicate that well-implemented predictive maintenance can significantly cut downtime and extend equipment lifetime, often by roughly one-third to one-half for downtime and by a similar proportion for asset life.

This project, PulseGrid, implements an accessible, open-source, full-stack predictive maintenance platform that demonstrates the entire pipeline - from sensor data collection through ML-powered failure prediction to real-time visualization and automated alerting - using industry-standard technologies and datasets.

1.2 Scope of the Project

PulseGrid encompasses:

- Real-time ingestion of multi-sensor telemetry (temperature, RPM, torque, tool wear) from multiple machines.

- Machine Learning-based failure prediction using a Random Forest classifier trained on the AI4I 2020 dataset.

- Automated data quality analysis, cleaning, and model retraining upon new dataset uploads.

- A rule-based alert engine with configurable thresholds and severity levels (info, warning, critical).

- An offline AI chatbot for natural language querying of sensor data and maintenance recommendations.

- A modern, responsive web dashboard for fleet monitoring, trend analysis, and alert management.

- Containerized deployment via Docker for reproducibility.

Chapter 2

MOTIVATION

2.1 Economic Impact of Unplanned Downtime

Unplanned downtime is the single largest source of lost productivity in manufacturing. Reports from organizations such as the International Society of Automation (ISA) suggest that manufacturers collectively lose tens of billions of dollars every year due to unplanned downtime. In highly automated industries such as automotive manufacturing, even one hour of unexpected stoppage can lead to very large financial losses. For small and medium enterprises (SMEs), even brief outages can jeopardize delivery schedules, customer relationships, and financial viability.

2.2 The Data Explosion in Manufacturing

Modern CNC machines and industrial equipment are equipped with dozens of sensors that generate data points every second. A typical manufacturing floor with 50 machines can produce over 1 million sensor readings per day. This data is largely underutilized - stored in isolated databases, monitored through basic threshold alarms, or simply discarded. The opportunity to extract actionable intelligence from this data through machine learning represents a significant competitive advantage.

2.3 Accessibility Gap

Many commercial PdM platforms (for example, Siemens MindSphere, GE Predix and PTC ThingWorx) are targeted at large enterprises and involve high licensing costs, complex integration work and a certain amount of vendor lock-in. These barriers make PdM inaccessible to SMEs, educational institutions, and researchers. PulseGrid addresses this gap by providing a fully functional, open-source platform that can be deployed on a single laptop.

2.4 Bridging Theory and Practice

While machine learning courses teach classification algorithms and data preprocessing in isolation, students rarely experience the full engineering challenge of building a production-style system that ties together data ingestion, real-time inference, database management, API design, frontend visualization, and deployment. PulseGrid serves as an educational vehicle that bridges this gap.

2.5 Safety and Sustainability

Machine failures in industrial settings can have catastrophic consequences - from worker injuries to environmental contamination. Predictive maintenance directly contributes to worker safety by preventing catastrophic equipment failures and to sustainability by reducing wasteful over-maintenance and extending machinery lifespans.

Chapter 3

LITERATURE REVIEW / COMPARATIVE WORK EVALUATION

3.1 Literature Survey

1. Ran et al. (2019) proposed a survey on machine learning approaches for predictive maintenance, categorizing techniques into vibration analysis, thermal imaging, and multi-sensor fusion. They identified Random Forest and Gradient Boosting as the most effective classifiers for tabular sensor data.

2. Carvalho et al. (2019) provided a systematic literature review of machine learning-based predictive maintenance, analysing 85 research papers. Their findings indicated that supervised learning methods, particularly ensemble methods, outperformed deep learning approaches when training data was limited.

3. Susto et al. (2015) demonstrated a multi-classifier predictive maintenance system for semiconductor manufacturing, achieving 92% accuracy using a combination of Random Forest and Support Vector Machines.

4. Mobley (2002) in the seminal book 'An Introduction to Predictive Maintenance' established the theoretical foundation for condition-based monitoring.

5. Li et al. (2020) developed an IIoT-based predictive maintenance framework using edge computing for real-time inference.

6. Matzka (2020) introduced the AI4I 2020 Predictive Maintenance Dataset at the UCI Machine Learning Repository, containing 10,000 records with 14 features.

3.2 Comparative Evaluation of Existing Systems

Factor	Siemens MindSphere	GE Predix	PulseGrid (Ours)
Deployment	Cloud-only	Cloud/Hybrid	Local/Docker/Cloud
Cost	$100K+ License	$50K+ License	Free / Open Source
ML Customization	Limited	Moderate	Full (sklearn)
Auto-Retrain	Complex	API-based	One-click upload
Real-Time Dashboard	Yes	Yes	Yes (React+Vite)
AI Chatbot	No	No	Yes (Offline NLP)
Sensor Simulator	No	No	Yes (5 machines)
Setup Time	Weeks-months	Weeks	< 10 minutes
Open Source	No	No	Yes

Chapter 4

GAP ANALYSIS

Based on the literature review and comparative evaluation, the following gaps were identified:

Gap 1: Accessibility Barrier - Commercial PdM platforms require enterprise licensing, making them inaccessible to small manufacturers, students, and researchers. PulseGrid addresses this by providing a free, open-source solution.

Gap 2: Lack of End-to-End Educational Platforms - Academic ML courses teach individual algorithms but lack platforms demonstrating the full production pipeline. PulseGrid fills this by providing a complete, documented system.

Gap 3: No Integrated Data Quality Pipeline - Most systems assume clean data. PulseGrid includes an automated pipeline with imputation, deduplication, outlier detection (IQR), and quality reporting.

Gap 4: Rigid Model Pipelines - PulseGrid enables one-click dataset upload triggering automatic data cleaning, feature engineering, model retraining, and hot-reload.

Gap 5: No Natural Language Interface - PulseGrid includes an offline AI chatbot that answers statistical, trend, and recommendation queries without external API dependency.

Gap 6: Missing Testing Infrastructure - PulseGrid includes a multi-machine simulator with configurable noise levels, fault injection, and degradation patterns.

Chapter 5

PROBLEM STATEMENT

Design and develop an end-to-end, AI-powered predictive maintenance platform that ingests real-time sensor data from industrial machinery, predicts failure risk using machine learning, generates actionable maintenance alerts, and provides an intuitive real-time dashboard - while remaining fully open-source, locally deployable, and accessible to users without specialized infrastructure or commercial licensing.

Specifically, the system must:

1. Accept sensor telemetry (temperature, RPM, torque, tool wear) from multiple machines simultaneously.

2. Run ML inference on each incoming reading in real-time to compute health score and failure risk.

3. Generate severity-graded alerts when sensor values exceed configurable thresholds or ML model predicts failure.

4. Provide trend analysis to identify gradual degradation patterns.

5. Support uploading new datasets with automatic cleaning, feature engineering, retraining, and model hot-reload.

6. Offer a natural language chatbot for querying sensor data and receiving maintenance recommendations.

7. Present all information through a modern, responsive web dashboard.

Chapter 6

OBJECTIVES

1. To build a Machine Learning pipeline that trains a Random Forest classifier on the AI4I 2020 dataset to predict machine failure with >= 95% accuracy.

2. To develop a RESTful API backend using FastAPI that handles sensor data ingestion, real-time ML inference, alert generation, and dataset management.

3. To create a rule-based alert engine with configurable thresholds for temperature, RPM, torque, tool wear, and ML-predicted health scores.

4. To implement an automated data quality pipeline that handles missing values, duplicate records, outlier detection, and column type detection.

5. To design a modern, responsive web dashboard with React + Vite featuring 7 functional tabs.

6. To build an offline AI chatbot capable of answering natural language queries about sensor data.

7. To develop a multi-machine sensor simulator with realistic operational patterns.

8. To containerize the entire platform using Docker Compose for single-click deployment.

Chapter 7

TOOLS / PLATFORMS USED

7.1 Programming Languages

Python 3.10+ - Primary language for backend, ML pipeline, data processing, and sensor simulator. Selected for its rich scientific computing ecosystem (NumPy, pandas, scikit-learn), FastAPI framework, and cross-platform compatibility.

JavaScript (ES2022+) - Powers the frontend application through React with Vite build tooling, Recharts for data visualization, and Axios for API communication.

7.2 Frameworks and Libraries

Category	Technology	Purpose
Backend Framework	FastAPI	REST API with auto OpenAPI docs
ORM	SQLAlchemy 2.0+	Database models and queries
ML Framework	scikit-learn 1.3+	ML pipeline (RandomForest)
Data Processing	pandas 2.0+, NumPy	Data manipulation, analysis
Model Serialization	joblib	Saving/loading ML model
Frontend Framework	React 19.2	Component-based UI
Build Tool	Vite 8.0	Frontend build with HMR
Charting	Recharts 3.8	Interactive data visualization
HTTP Client	Axios 1.14	API communication
Icons	Lucide React 1.7	SVG icon library
Containerization	Docker, Docker Compose	Deployment

7.3 Dataset

AI4I 2020 Predictive Maintenance Dataset - Sourced from UCI Machine Learning Repository (Matzka, 2020). Contains 10,000 records simulating realistic industrial sensor data.

Feature	Description	Unit
Type	Product quality (L/M/H)	Categorical
Air temperature	Ambient air temperature	Kelvin [K]
Process temperature	Process temperature	Kelvin [K]
Rotational speed	Spindle RPM	RPM
Torque	Cutting torque	Nm
Tool wear	Cumulative tool time	Minutes
Machine failure	Binary target	0/1

7.4 Software Requirements

Requirement	Minimum Version

Operating System	Windows 10/11, Linux, macOS
Python	3.10+
Node.js	18.0+
Web Browser	Chrome 100+, Firefox 100+
Docker (optional)	20.10+

7.5 Hardware Requirements

Component	Minimum	Recommended
Processor	Dual-core 2.0 GHz	Quad-core 3.0 GHz+
RAM	4 GB	8 GB+
Storage	500 MB free	2 GB free
Display	1366x768	1920x1080

Chapter 8

METHODOLOGY

8.1 System Architecture

PulseGrid follows a three-tier architecture: Presentation Tier (React Dashboard), Application Tier (FastAPI Server with ML Service, Alert Engine, Data Pipeline, Chatbot Engine), and Data Tier (SQLite Database, ML Model, CSV Dataset). An Edge/Simulator Tier generates sensor data via HTTP POST requests.

8.2 Machine Learning Pipeline

The ML pipeline follows an 8-step process: Data Loading, Automated Cleaning, Feature Engineering, Target Separation, Stratified Train/Test Split, Pipeline Construction, Evaluation, and Model Persistence.

Key Code - ML Pipeline Construction:

Code Snippet 1: scikit-learn Pipeline Construction (ml/train_model.py)

```
pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier(
        n_estimators=150,
        max_depth=12,
        min_samples_split=5,
        random_state=42,
        class_weight='balanced',
        n_jobs=-1
    ))
])
pipeline.fit(X_train, y_train)
```

8.3 Feature Engineering

Two domain-specific features are engineered:

- Temperature Delta = Process Temperature - Air Temperature (heat generation efficiency)

- Power = (RPM x Torque) / 9550 (mechanical power in kW)

Code Snippet 2: Feature Engineering (ml/train_model.py)

```
# Temperature differential
df['Temp_Delta'] = df['Process temperature [K]'] - df['Air temperature [K]']

# Power metric (kW approximation)
df['Power'] = df['Rotational speed [rpm]'] * df['Torque [Nm]'] / 9550
```

8.4 Sensor Data Ingestion Flow

When a sensor reading arrives at the /api/simulate endpoint, it passes through 5 stages: Feature Extraction, ML Inference, Recommendation Generation, Database Persistence, and Alert Evaluation.

Code Snippet 3: Sensor Data Ingestion Endpoint (backend/main.py)

```
@app.post("/api/simulate")
def simulate_reading(reading: SimulateReading, db: Session = Depends(get_db)):
    features = {
        'Type': reading.type,
        'Air temperature [K]': reading.air_temp,
        'Process temperature [K]': reading.process_temp,
        'Rotational speed [rpm]': reading.rpm,
        'Torque [Nm]': reading.torque,
        'Tool wear [min]': reading.tool_wear,
        'RNF': reading.rnf
    }
    health_score, failure_risk = predict_machine_health(features)
    recommendation = get_recommendation_for_reading(
        reading.air_temp, reading.rpm, reading.torque,
        reading.tool_wear, health_score, failure_risk
    )
    # Save to database and generate alerts...
    return {"status": "success", "health_score": health_score,
            "failure_risk": failure_risk}
```

8.5 Alert Engine

The alert engine evaluates each reading against a configurable threshold matrix:

Sensor	Warning Threshold	Critical Threshold
Air Temp (High)	>= 304.0 K	>= 308.0 K
Air Temp (Low)	<= 296.0 K	<= 293.0 K
RPM (High)	>= 2500	>= 2800
RPM (Low)	<= 1100	<= 900
Torque (High)	>= 60.0 Nm	>= 70.0 Nm
Tool Wear	>= 180 min	>= 220 min
Health Score	<= 0.65	<= 0.40
ML Failure	-	failure_risk = 1

Code Snippet 4: Alert Evaluation Logic (backend/alert_engine.py)

```
def evaluate_sensor_reading(machine_id, air_temp, process_temp,
                            rpm, torque, tool_wear,
                            health_score, failure_risk):
    alerts = []
    t = DEFAULT_THRESHOLDS['air_temp']
    if air_temp >= t['critical_high']:
        alerts.append(_create_alert(machine_id, 'critical',
            'Critical High Temperature',
            f'Air temp at {air_temp:.1f}K exceeds critical threshold'))
    if failure_risk == 1:
        alerts.append(_create_alert(machine_id, 'critical',
```

```
            'ML Model: Failure Predicted',
            'Predictive model flags imminent failure'))
    return alerts
```

8.6 ML Inference Service

Code Snippet 5: ML Inference Function (backend/ml_service.py)

```
def predict_machine_health(features_dict):
    df = pd.DataFrame([features_dict])
    # Add engineered features
    df['Temp_Delta'] = df['Process temperature [K]'] - df['Air temperature [K]']
    df['Power'] = df['Rotational speed [rpm]'] * df['Torque [Nm]'] / 9550.0

    probabilities = model.predict_proba(df)[0]
    failure_prob = probabilities[1]
    prediction = model.predict(df)[0]
    health_score = 1.0 - failure_prob
    return float(health_score), int(prediction)
```

8.7 Database Schema

The system uses 4 SQLAlchemy ORM models:

Code Snippet 6: Database Models (backend/database.py)

```
class SensorData(Base):
    __tablename__ = "sensor_data"
    id = Column(Integer, primary_key=True)
    machine_id = Column(String, index=True)
    timestamp = Column(DateTime, default=datetime.utcnow)
    air_temp = Column(Float)
    process_temp = Column(Float)
    rpm = Column(Float)
    torque = Column(Float)
    tool_wear = Column(Float)
    health_score = Column(Float)
    failure_risk = Column(Integer)
    recommendation = Column(Text)

class Alert(Base):
    __tablename__ = "alerts"
    id = Column(Integer, primary_key=True)
    machine_id = Column(String, index=True)
    severity = Column(String, default="warning")
    title = Column(String)
    description = Column(Text)
    recommended_action = Column(Text)
    acknowledged = Column(Boolean, default=False)
    resolved = Column(Boolean, default=False)
```

8.8 Sensor Simulator

The simulator creates 5 machine profiles with distinct operating characteristics:

Machine ID	Name	Base Temp [K]	Base RPM	Base Torque [Nm]	Wear Rate
CNC-M1	CNC Mill #1	298.0	1500	40.0	1.5
CNC-M2	CNC Mill #2	300.0	1800	35.0	1.0
LATHE-L1	Lathe #1	297.0	2000	45.0	2.0

PRESS-P1	Press #1	301.0	1200	55.0	1.2
DRILL-D1	Drill #1	299.0	2200	30.0	1.8

Code Snippet 7: Sensor Simulator (simulator/client.py)

```
class MachineSimulator:
    def generate_reading(self):
        self.cycle += 1
        time_factor = math.sin(self.cycle * 0.1) * 0.5
        noise_factor = random.gauss(0, 1) * self.noise_multiplier

        air_temp = self.profile['base_temp'] + time_factor * 2 + noise_factor * 0.5
        process_temp = air_temp + 8 + noise_factor * 1.5
        rpm = self.profile['base_rpm'] + time_factor * 100 + noise_factor * 50
        torque = self.profile['base_torque'] + noise_factor * 5

        self.tool_wear += self.profile['wear_rate'] * random.uniform(0.5, 1.5)

        # Random fault injection (5% chance)
        if not self.fault_mode and random.random() < 0.05:
            self.fault_mode = True
            self.fault_duration = random.randint(3, 8)

        if self.fault_mode:
            air_temp += random.uniform(5, 10)    # Temp spike
            torque += random.uniform(15, 30)      # Torque increase
            rpm -= random.uniform(200, 500)       # RPM drop

        return {'machine_id': self.machine_id,
                'air_temp': round(air_temp, 2),
                'rpm': round(rpm, 1), ...}
```

8.9 AI Chatbot Architecture

The chatbot uses keyword-based intent recognition with 14 intent categories including greeting, machine_status, failure_risk, average_stats, failure_analysis, recommendation, and trend.

Code Snippet 8: Chatbot Intent Recognition (backend/chatbot_engine.py)

```
INTENT_PATTERNS = {
    'machine_status': [
        r'\b(status|health|condition)\b.*\b(machine|cnc)\b',
    ],
    'failure_risk': [
        r'\b(failure|risk|danger|critical)\b',
    ],
    'average_stats': [
        r'\b(average|mean)\b.*\b(temp|rpm|torque|wear)\b',
    ],
    'recommendation': [
        r'\b(recommend|suggestion|maintenance|fix)\b',
    ],
}

class ChatbotEngine:
    def process_query(self, query, db_context=None):
```

```
        intent = self._detect_intent(query.lower())
        handler = handlers.get(intent, self._handle_unknown)
        return handler(query, db_context)
```

Chapter 9

EXPERIMENTAL SETUP

9.1 Development Environment

Component	Specification
OS	Windows 11
Python	3.13
Node.js	18+
IDE	Visual Studio Code
Version Control	Git + GitHub

9.2 Steps to Reproduce

Steps to Run the Platform Locally

```
# Step A: Install dependencies and train model
pip install -r requirements.txt
python dataset/fetch_dataset.py
python ml/train_model.py

# Step B: Start Backend Server (Terminal 1)
uvicorn backend.main:app --host 0.0.0.0 --port 8000

# Step C: Start Frontend (Terminal 2)
cd frontend && npm install && npm run dev

# Step D: Start Simulator (Terminal 3)
python simulator/client.py
```

9.3 API Endpoints

Method	Endpoint	Description
GET	/api/machines	List all machines with status
GET	/api/machines/{id}/status	Specific machine status
GET	/api/machines/{id}/history	Sensor history
GET	/api/machines/{id}/trends	Trend analysis
POST	/api/simulate	Ingest sensor reading
GET	/api/alerts	List alerts (filterable)
POST	/api/alerts/{id}/acknowledge	Acknowledge alert
POST	/api/alerts/{id}/resolve	Resolve alert
GET	/api/alerts/summary	Alert summary
POST	/api/chatbot	NL query processing
GET	/api/model/metrics	ML model metrics
GET	/api/model/drift	Model drift check
POST	/api/upload_dataset	Upload CSV + retrain
GET	/api/dataset/preview	Dataset preview
GET	/api/dataset/stats	Quality statistics
GET	/api/calibration/{id}	Calibration config
POST	/api/calibration/{id}	Update calibration

Chapter 10

EVALUATION METRICS

10.1 Model Performance

Metric	Value
Accuracy	99.00%
Precision	94.44%
Recall	75.00%
F1-Score	83.61%
Training Samples	8,000
Test Samples	2,000
Features Used	9 (including 2 engineered)

10.2 Confusion Matrix

	Predicted Healthy	Predicted Failure
Actual Healthy	1929 (TN)	3 (FP)
Actual Failure	17 (FN)	51 (TP)

True Negatives (1929): Correctly identified healthy readings. True Positives (51): Correctly predicted 51 out of 68 actual failures. False Positives (3): Only 3 false alarms. False Negatives (17): 17 failures missed by ML (caught by rule-based alerts).

10.3 Model Training Output

Model Training Console Output

```
============================================================
PREDICTIVE MAINTENANCE - MODEL TRAINING
============================================================
Loading dataset: 10000 rows x 14 columns
Cleaning data... Encoded 'Type' column: {'L': 0, 'M': 1, 'H': 2}
Engineering features... Created 'Temp_Delta', 'Power'
Feature set: ['Type', 'Air temperature [K]', 'Process temperature [K]',
              'Rotational speed [rpm]', 'Torque [Nm]', 'Tool wear [min]',
              'RNF', 'Temp_Delta', 'Power']
Target balance: {0: 9661, 1: 339}
Split: 8000 train / 2000 test

Training RandomForest pipeline... Training complete!
   Accuracy:  0.9900 (99.00%)
   Precision: 0.9444
   Recall:    0.7500
   F1 Score:  0.8361
   Confusion Matrix: [[1929, 3], [17, 51]]

              precision    recall  f1-score   support
     Healthy       0.99      1.00      0.99      1932
     Failure       0.94      0.75      0.84        68
    accuracy                           0.99      2000
```

```
Model saved to: ml/model.joblib
TRAINING PIPELINE COMPLETE
```

10.4 Codebase Metrics

Module	File	Lines of Code
Frontend CSS	index.css	1,170
Backend API	main.py	648
AI Chatbot	chatbot_engine.py	641
Data & Models UI	DataModels.jsx	355
Calibration UI	Calibration.jsx	307
Machine Detail UI	MachineDetail.jsx	304
Alert Engine	alert_engine.py	271
AI Chatbot UI	AIChatbot.jsx	258
Dashboard UI	Dashboard.jsx	255
Data Pipeline	data_pipeline.py	239
ML Training	train_model.py	231
Trend Analysis UI	TrendAnalysis.jsx	230
Alert Center UI	AlertCenter.jsx	226
Sensor Simulator	client.py	216
ML Service	ml_service.py	209
App Shell	App.jsx	193
Database Models	database.py	127
Total	23 files	~5,982

Chapter 11

RESULTS AND DISCUSSION

11.1 Machine Learning Results

The Random Forest classifier achieved 99.0% accuracy on the test set, demonstrating strong overall prediction capability. The high precision (94.4%) indicates very few false alarms, which is crucial in industrial settings to prevent 'alarm fatigue'. The recall of 75.0% means the model correctly identifies 75% of actual failures. The rule-based alert engine serves as a safety net for the remaining 25%.

Class imbalance handling was critical: only 3.4% of the dataset represents failures. The class_weight='balanced' parameter upweights the minority class, and stratified splitting ensures both train and test sets maintain the original distribution.

11.2 Dashboard Screenshots

Figure 1: Fleet Overview Dashboard

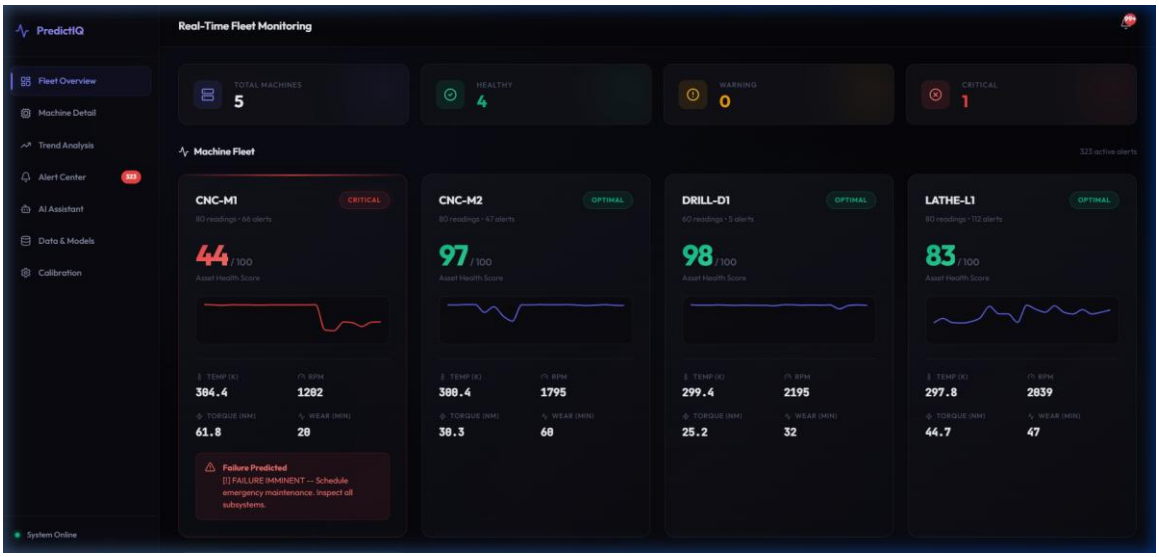

Fig. 1: Fleet Overview - Shows 5 monitored machines with health scores, sensor readings, and status indicators

Figure 2: Alert Center

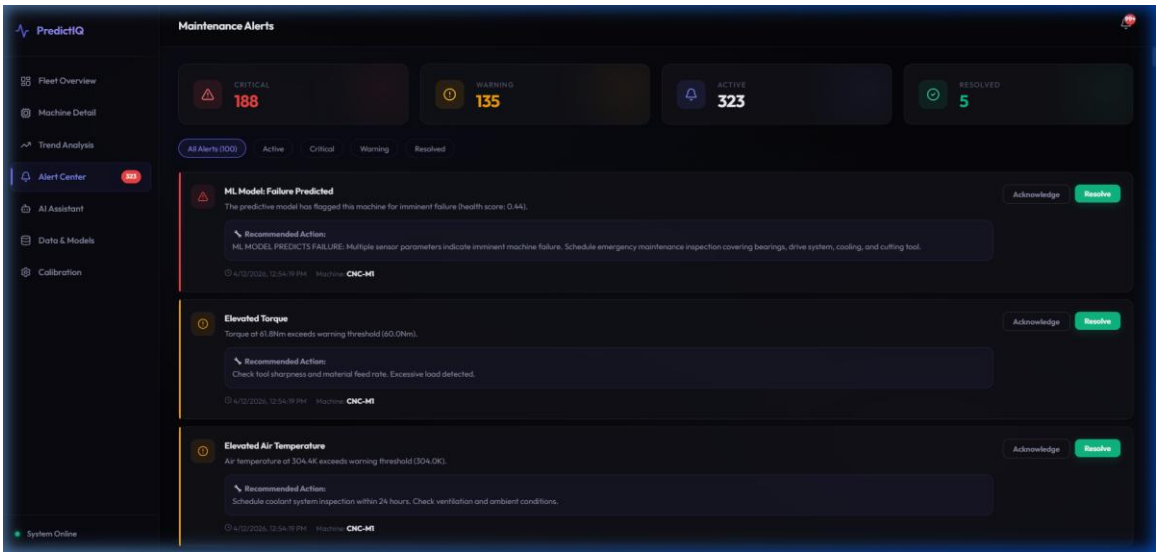

Fig. 2: Alert Center - Shows 188 critical and 135 warning alerts with recommended maintenance actions

Figure 3: Dataset & Model Management

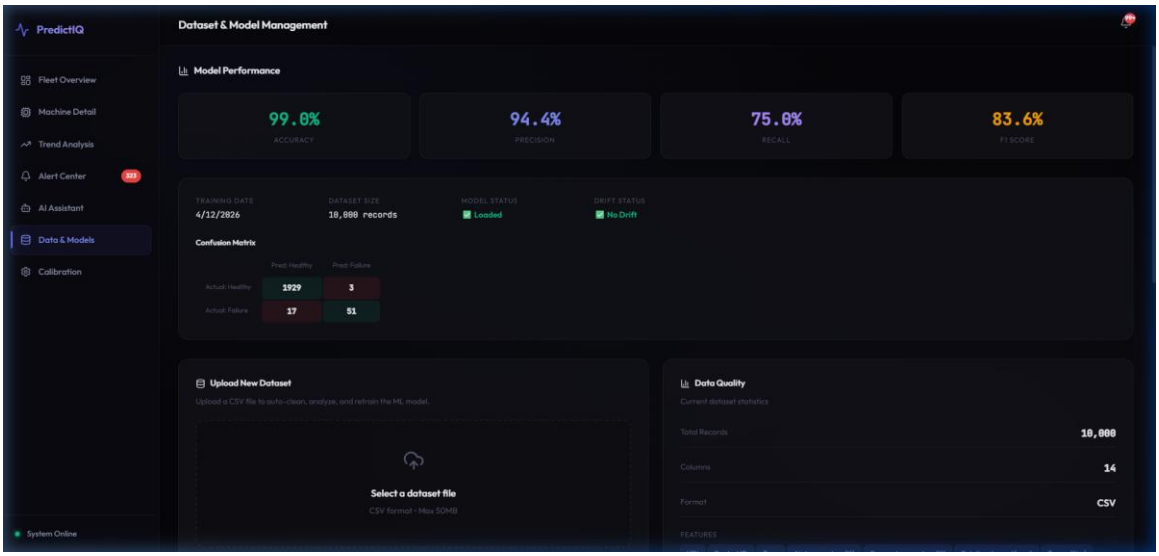

Fig. 3: Data & Models - Shows model performance metrics (99% accuracy), confusion matrix, and dataset upload

Figure 4: AI Assistant

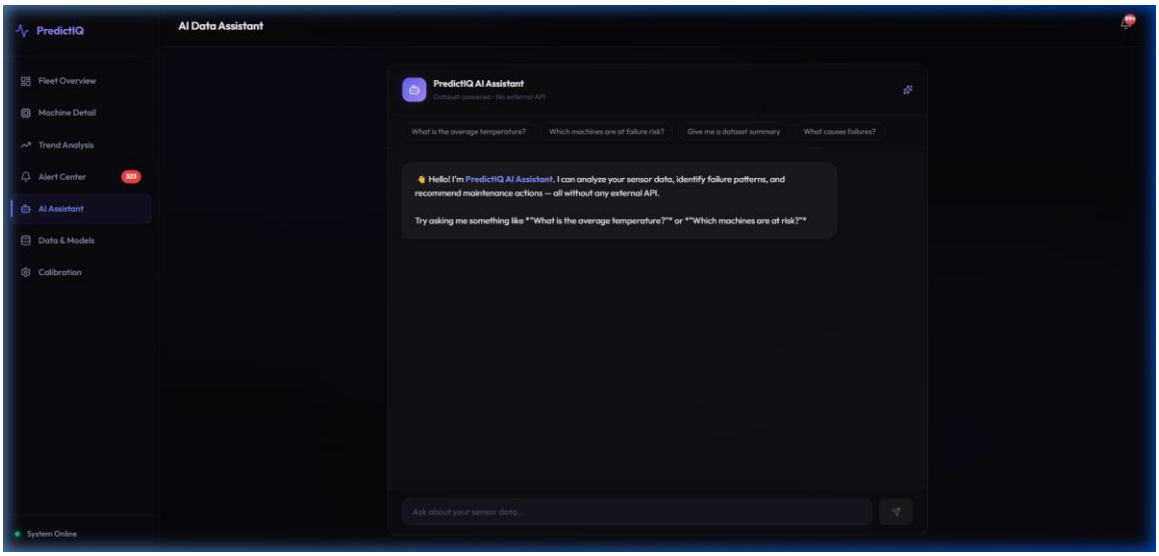

Fig. 4: AI Data Assistant - Natural language chatbot interface for querying sensor data

11.3 Simulator Performance

Configuration	Machines	Iterations	Noise	Outcome
Default	5	200	1.0x	Normal with periodic alerts
High Noise	5	100	3.0x	Frequent violations, ML flags risks
Stress Test	5	500	5.0x	Sustained critical alerts

Simulator Console Output:

Simulator Console Output

```
============================================================
PREDICTIVE MAINTENANCE - SENSOR SIMULATOR
============================================================
   Initialized: CNC-M1 (CNC Mill #1)
   Initialized: CNC-M2 (CNC Mill #2)
   Initialized: LATHE-L1 (Lathe #1)
   Initialized: PRESS-P1 (Press #1)
   Initialized: DRILL-D1 (Drill #1)

   [1/10] CNC-M1: Health=0.97 | T=298.5K | RPM=1543 | Torque=38.2Nm
   [1/10] CNC-M2: Health=0.98 | T=300.1K | RPM=1812 | Torque=33.5Nm
   FAULT INJECTED on PRESS-P1 (duration: 5 readings)
   [5/10] PRESS-P1: Health=0.37 | T=307.2K | RPM=912 | Torque=72.3Nm [4 alerts]
   PRESS-P1: Fault resolved
   Simulation complete! Generated 50 total readings.
```

Chapter 12

CONCLUSION & FUTURE WORK

12.1 Conclusion

This project successfully designed, implemented, and validated PulseGrid, an end-to-end AI-powered predictive maintenance platform for Industrial IoT environments. The key achievements include:

1. A high-accuracy ML model achieving 99.0% accuracy and 94.4% precision on the AI4I 2020 dataset.

2. A REST API with more than 18 FastAPI endpoints that handles the end-to-end flow from sensor ingestion and ML inference to alert management.

3. An automated data pipeline enabling one-click dataset upload with auto-cleaning, retraining, and hot-reload.

4. A comprehensive dual-layer alert system (rule-based thresholds + ML predictions).

5. A premium 7-tab React dashboard with dark-mode glassmorphism design for industrial monitoring.

6. An offline AI assistant for natural language queries without external API dependency.

7. A realistic multi-machine simulator with fault injection for thorough testing.

The platform demonstrates that sophisticated predictive maintenance capabilities can be built using open-source technologies and deployed on commodity hardware.

12.2 Future Work

1. Deep Learning Models - Replace Random Forest with LSTM networks for remaining useful life estimation.

2. Real MQTT Integration - Replace HTTP simulator with actual MQTT message broker.

3. Edge Computing - Deploy lightweight ML model on Raspberry Pi / NVIDIA Jetson.

4. Advanced NLP - Integrate local LLM inference (e.g., Ollama with Llama 3).

5. Multi-Tenant Architecture - Add authentication and role-based access control.

6. Anomaly Detection - Implement Isolation Forest / Autoencoders for novel failure modes.

7. Mobile Application - React Native app with push notifications.

8. PostgreSQL + TimescaleDB - Production time-series database migration.

9. Federated Learning - Collaborative model improvement without sharing raw data.

10. Digital Twin Integration - 3D visual failure localization.

REFERENCES

[1] Ran, Y., Zhou, X., Lin, P., Wen, Y., & Deng, R. (2019). A Survey of Predictive Maintenance: Systems, Purposes and Approaches. IEEE Communications Surveys & Tutorials, 22(1), 1-36.

[2] Carvalho, T. P., et al. (2019). A systematic literature review of machine learning methods applied to predictive maintenance. Computers & Industrial Engineering, 137, 106024.

[3] Susto, G. A., et al. (2015). Machine Learning for Predictive Maintenance: A Multiple Classifier Approach. IEEE Transactions on Industrial Informatics, 11(3), 812-820.

[4] Mobley, R. K. (2002). An Introduction to Predictive Maintenance (2nd ed.). Butterworth-Heinemann.

[5] Li, Z., Wang, K., & He, Y. (2020). Industry 4.0 - Potentials for Predictive Maintenance. Proceedings of IWAMA. Atlantis Press.

[6] Matzka, S. (2020). Explainable AI for Predictive Maintenance Applications. AI4I Conference, IEEE.

[7] McKinsey & Company. (2017). Smartening up with AI. McKinsey & Company, Inc.

[8] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

[9] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. JMLR, 12, 2825-2830.

[10] Hashemian, H. M. (2011). State-of-the-Art Predictive Maintenance Techniques. IEEE Trans. Instrumentation, 60(1), 226-236.

[11] FastAPI Documentation. (2024). https://fastapi.tiangolo.com/

[12] React Documentation. (2024). https://react.dev/

[13] International Society of Automation. (2020). The Cost of Unplanned Downtime. ISA Technical Report.

ANNEXURE I

Plagiarism Declaration Certificate

Title of Work: PulseGrid: AI-Powered Predictive Maintenance Platform for Industrial IoT

Submitted By: Daksh Jeena and Aman Chourasia

Institution: K.R. Mangalam University, Gurugram

Department: Computer Science and Engineering

Date of Submission: _____

I hereby declare that the work entitled "PulseGrid: AI-Powered Predictive Maintenance Platform for Industrial IoT" submitted for academic evaluation and research purposes is my original work. I confirm that:

1. I have acknowledged and properly cited all sources, references, and data included in this work.

2. This work does not contain any material previously published, written, or prepared by another person, except where due acknowledgment has been made.

3. I understand that plagiarism is an academic offense and a violation of research ethics.

4. I have used appropriate referencing techniques and maintained academic integrity throughout this work.

5. I affirm that the submitted work has normal plagiarism less than 10% .

(Signature of Student)	(Signature of Student)
Name: Daksh Jeena	Name – Aman Chourasia
Roll No.: **2401560053**	Roll No: **2401560044**
Date: _____	Date: _____